

P?S/8 Tiny File System [TFS] Internal Structures

Last edit: 13-Aug-2015 cjl

The basic P?S/8 user file structure is the TFS directory. The keyboard monitor and embedded editor includes a variety of commands to edit text files stored in the TFS directory as well as manage files within the directory [regardless of format].

Every TFS file is exactly 2048 words long stored in 16 adjacent logical blocks of 128 12-bit words each. P?S/8 expects the directory to have several internal values to allow proper operation as well as entries for the particular files.

TFS filenames are up to six characters where each character is either a letter, number or certain specific non-alphanumeric characters such as \, ^, [, and]. Some users prefer to indicate a programming project's files by the use of pseudo-sequence numbers such as PRG[1], PRG[2], PRG[3], PRG[4], PRG[5], PRG[6]. [Note: Any P?S/8 system program can be passed as many as 17 files with some restrictions in certain cases (such as the P?S/8 PAL assembler that generally also requires binary output files; the list of all files must be 17 files or less).]

The internal entry for any TFS file always occupies three adjacent 12-bit words somewhere within blocks 0015-0016. Figure 1 illustrates a few file entries in a minimally-used directory.

At first, the entire directory was empty; then the files TCBIN, TCBOOT, FOO and BAR were entered; then FOO was deleted creating what is usually referred to as a free slot. [Note: The additional slots past those actively in the directory are referred to as available slots; however, from the vantage point of allocation, the only difference is a free slot had been occupied while an available slot apparently has no prior usage history. If required, the keyboard monitor EN[ter] command can be used to create tentative entries for the purpose of manual attempts at file recovery; when files are deleted in perfect reverse order, free slots are converted back to available slots.]

TCBIN is five characters, the trailing six bits in the name entry for TCBIN is 00. TCBOT is a six characters, the three words are a complete file name. Since FOO was deleted, the first two characters are replaced by the value 7777. The rest of the entry still retains R followed by three null characters. BAR is six characters and still active, there are three sets of six bits past the name of 00 to fill the rest of the three words.

BLOCK: 0:0015										
0000	2403	0211	1600	2403	0217	1724	7777	1700	*TCBIN@TCBOOT??0@*	
0010	0000	0201	2200	0000	0000	0000	0000	0000	*@BAR@@@@@@@@@@@@*	
0020	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0030	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0040	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0050	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0060	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0070	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0100	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0110	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0120	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0130	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0140	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0150	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0160	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	
0170	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@*	

Figure 1: Dump of Block 0015 of P?S/8 Version 8Z for TC08.

Figure 2 shows the last three words of the TFS directory set to the specific values particular to the standard release of P?S/8 Version 8Z for the TC01/TC08 DECTape. System programs are allocated starting in very low blocks [after certain hard-wired data blocks reserved for various purposes including images of non-system device handlers]. In the standard release of P?S/8 for TC01/TC08 DECTape, all system programs are stored in blocks ending at block 0735. [Additional blocks 0736 through block 0777 are reserved for future expansion of the system program area.]

word[177] of block 0016 contains the starting block for the first file entered into the TFS directory. In the stated example, this is block 1000. As such, the file TCBIN occupies blocks 1000-1017. The file TCB00T occupies blocks 1020-1037. The deleted file [formerly known as F00, but now the only information is that it was three characters and the third character is R] occupies blocks 1040-1057. The file BAR occupies blocks 1060-1077. All blocks from 1100 through 2677 are unallocated as part of a series of available slots.

word[176] of block 0016 contains the high-limit block. The highest block of the final file allowed in the directory must end just prior to this block. In this example, the highest file allowed would occupy blocks 2660-2677.

word[175] is a more recently added feature of the TFS directory. The information is essentially the same as the high-limit block [given the value of the starting block], but expressed in an alternate form used by internal routines within the keyboard monitor to streamline the coding.

Within the keyboard monitor, the image of the TFS directory is stored in 07000-07377. Instead of calculating the block of a tentative file and comparing to the high-limit block, this information is the address in memory of the first directory slot not allowed.

BLOCK: 0:0016											
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0010	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0020	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0030	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0040	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0050	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0060	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0070	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0100	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0110	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0120	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0130	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0140	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0150	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0160	0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@*	
0170	0000	0000	0000	0000	0000	0514	2700	1000		*@@@@@@@@@ELW@H@*	

Figure 2: Dump of Block 0016 of P?S/8 Version 8Z for TC08.

In the example given, the number of slots allowed would be 2700-1000/20 or 74 files allowed. [Note: All examples are stated in terms of octal values; 74 octal files can also be expressed as 60 decimal files.]

Since each file entry is three words, 3x74 files or 264 words of the directory can be used to represent files for this device. Since the directory is loaded into 07000-onward, the first entry address disallowed would be 7264. The value in word[175] is expressed in two's complement form which is 0514. Thus, if a TFS entry is attempted, it would be rejected if it was calculated as 7264 [or higher] within the directory address range.

For larger devices, additional file slots are allowed, up to the maximum capacity of 124 [84 decimal] files occupying 2500 blocks. Extremely spartan DECTape-based storage units could allow this much file usage by eliminating virtually all system programs. [Note: For use primarily as a secondary tape unit, this can easily be accomplished; however, there are presently no utilities to automatically implement such a reconfiguration of resources. With a proper understanding of the TFS and system programs directory, this can be accomplished by an advanced user using either P?S/8 DUMP or P?S/8 BLKODT. Future releases of P?S/8 will include reconfiguration utilities to allow all users the ability to adjust file allocation storage.]

File block boundary considerations.

While there are no internal requirements for any blocks within a TFS directory to be on any particular boundary, P?S/8 execution commands require that all files be aligned to boundaries of eight blocks. This restriction exists to allow P?S/8 to communicate the starting block and logical unit of any file in 12-bit words. When the restriction is obeyed, the block number can be expressed in the high-order nine bits of the 12-bit word while the logical unit can be expressed in the low-order three bits of the same 12-bit word. Lists of such words are passed to system programs that will unpack the passed values accordingly.

Very early P?S/8 system devices did not impose the boundaries stated here; the logical unit concept had not yet been implemented. It is not likely any of these early systems exist presently; however, the forerunner system known as the R-L Monitor System [or MS/8] for TC01/TC08 DECTape was submitted to DECUS and in theory was used by certain individuals who are likely candidates for upgrade to P?S/8.

The R-L Monitor System also uses the TFS directory format and is compatible with the P?S/8 Version 8Z keyboard monitor when used as a non-system logical device unit for file access [read purposes] only. Files in binary format can be copied directly as the R-L Monitor binary file format is a proper subset of P?S/8 binary files [lacking support for extended memory]. Text files require conversion using the P?S/8 stand-alone RLFIX utility [available from the author of this document].

It is recommended the R-L Monitor System tape be backed up before upgrading, as the RLFIX program modifies the supplied DECTape; all converted text files will no longer be compatible with the R-L Monitor System; the conversion process is not reversible. The selected files will be upgraded to P?S/8 text format and stored back in-place with no loss of data. After the upgrade process has completed, all selected text [and binary] files will be fully compatible with P?S/8 standards; however, the TFS directory will not be in compliance with the boundary requirement needed for full access by P?S/8 programs.

As such, the files will require transfer to another P?S/8 tape. [Note: The R-L Monitor System only supports TC01/TC08 DECTape; as such, the conversion must be accomplished using the TC01/TC08 release of P?S/8.] The file transfer can be accomplished with keyboard monitor commands such as FE[tch] filnam:1 followed by WR[ite] filnam:0 or similar. Alternatively, the P?S/8 FILMAN utility can be used to allow files to be copied in groups using the wild-card match feature of FILMAN; file types can be restricted at the same time if desired. [Additional information about P?S/8 FILMAN is available in other documents.]

Once the files have been upgraded and transferred to boundary-conforming media, the files can be used with all relevant P?S/8 utilities.

[end-of-file]